3 Nov · page 1 of 2 – app01 from power-on: disk, RAM, kernel, systemd and mounts

green = on the disk (stays) · purple/orange = in RAM (gone at power-off) · teal = on another server · ① ② ③ = the order at boot

DISK of app01 – everything that STAYS

/boot — start files

└ vmlinuz	the kernel
└ initrd.img	helper with disk drivers
└ grub/	boot menu (not copied)

/etc — all settings = the "recipes"

└ fstab	which folder → which source
└ systemd/system/	services, timers, limits
└ systemd/system.control/	saved by set-property
└ netplan/	IP addresses
└ ssh/	SSH settings
└ passwd · group	users and groups
└ hostname	the server name

/usr — the programs

└ bin/	bash, ls, nano, stress-ng …

/home — users' own files

└ bilal_002/	my files

/var — data that grows

└ log/	logs (journal)
└ lib/	databases, Docker data

① copy

② read

RAM – the working space (empty at power-on)

KERNEL – the manager

copied from /boot at power-on

starts systemd as the first program (PID 1)

PAGE CACHE – copies of files that were read

/etc/fstab, unit files, bash, ls …

only a copy for speed – original stays on the disk

③ build

LIVE THINGS – built by systemd from the recipes

- mount table (folder → source)
- running services and processes (PIDs)
- cgroup boxes + limits

built again at every boot

/run – real files, but kept in RAM

/run/user/1000	my private folder, made at login
/run/credentials/…	secrets for one service
/run/snapd/ns	snap app isolation
/run/systemd/…	--runtime limits (gone at reboot)

KERNEL VIEWS – not real files, live information

/proc processes /dev devices

/sys hardware + /sys/fs/cgroup (my limit boxes)

stor01

(another server)

its own DISK

/backups

shared over the network (NFS)

STAYS

Who asks the kernel for a file? – every program, directly

systemd (PID 1) — starts services

nginx — a service

bash · ls · cat — my commands

KERNEL: "Is a copy already in RAM?"

yes → gives the copy at once (fast)

no → reads the disk, keeps a copy, then gives it

DISK — only when needed

the program just waits

One folder tree – every folder is connected (mounted) to a source, and the SOURCE decides if it stays

/	DISK	stays
/etc /home /usr /var /boot	DISK	stays
/run /run/user/1000 …	RAM	gone at reboot
/proc /sys /dev /run/rpc_pipefs	KERNEL VIEW	rebuilt every boot
/mnt/… (backups)	stor01 over NFS	stays on stor01

The list "folder → source" = the MOUNT TABLE (RAM)

its recipe: /etc/fstab (disk)

see it live: findmnt

saving a file: the folder's source decides → disk folder = stays · /run = gone at reboot · NFS folder = stays on stor01

3 Nov · page 2 of 2 – live test vs permanent, and my memory-limit test on app01

the big idea of the day: the work always happens in RAM – WHERE you write it decides if it stays

A · Every setting has two ways: LIVE (a test, gone at reboot) or a FILE in /etc (stays)

the change	LIVE – works at once · gone at reboot	FILE on the disk – read at every boot · stays
IP forwarding (gw01)	`sysctl -w net.ipv4.ip_forward=1 (/proc/sys/…)`	`/etc/sysctl.conf`
a service	`systemd-run --unit=mytest …`	`/etc/systemd/system/mytest.service + enable`
a limit	`systemd-run -p MemoryMax=… · set-property --runtime`	`set-property · …service.d/limits.conf`
a mount	`mount 192.168.60.20:/backups /mnt`	`/etc/fstab`
an IP address	`ip addr add …`	`/etc/netplan/…yaml`
a saved file	`– (saving in /home, /etc … already goes to disk)`	`any folder on the disk = stays`

admin habit: ① try it LIVE → ② check it works → ③ write it into the FILE so it stays

/proc /sys /dev /run = see and change the system live (kernel views + RAM) – good for trying first · /etc = the settings programs read at start

B · The test pattern – systemd-run makes a temporary box, -p puts a limit on it, stress-ng makes fake load

```
sudo systemd-run --scope -p <LIMIT> stress-ng <LOAD> --timeout <TIME>
```

`--scope`

box for MY command: output on my screen, deleted when the command ends

`-p <LIMIT>`

MemoryMax = kill · MemoryHigh = slow + swap

CPUQuota = throttle · TasksMax = max processes

`stress-ng <LOAD>`

`--vm 1 --vm-bytes 300M` = 1 worker 300MB

`--cpu 1` = busy core · `--fork 4` = forks

where does the box go?

with sudo → the system systemd → system.slice (the default place)

C · My test on app01: MemoryMax=100M, no swap, stress-ng wants 300M → killed inside its box only

1. **systemd-run** – temporary box – `run-ra9665….scope`
2. **stress-ng worker** – wants ~340 MB – `total-vm 349924kB`
3. **hits the wall** – real RAM ~96 MB – `anon-rss 98792kB`
4. **OOM killer** – kills the worker – `7× · new PID each time`
5. **parent survives** – PID 2687 – `"successful run completed"`
6. **box deleted** – command ended – `nothing left behind`

proof in the journal: constraint=CONSTRAINT_MEMCG · oom_memcg=/system.slice/run-ra9665….scope

CONSTRAINT_MEMCG = killed because of the BOX limit, not because app01 ran out of RAM · only that box was touched – services next to it kept running

same idea as a Kubernetes pod "OOMKilled" or Docker --memory

D · Useful checks

`pgrep -a stress-ng`	is a test still running?
`systemctl list-units --all | grep mytest`	old named test boxes?
`sudo pkill stress-ng · systemctl stop mytest`	stop leftovers
`sudo systemctl reset-failed`	clean old failed boxes
`sudo journalctl -k --since "5 min ago" | grep -i killed`	OOM kills in the kernel log
`findmnt · free -h · cat /etc/fstab`	mounts · RAM/swap · mount recipe

Final paragraph – the whole day in one place

Everything is stored on the disk, and RAM starts empty. At power-on the kernel (vmlinuz with its helper initrd.img) is copied from /boot into RAM and becomes the manager; it starts systemd as PID 1. systemd reads its recipes in /etc – each read puts a copy of that one file in RAM (page cache) while the original stays on the disk – and from them builds the live things in RAM: the mount table, the services and the cgroup boxes. Every program asks the kernel for files; the kernel gives a copy from RAM, or reads the disk first. Each folder is mounted to a source: disk folders stay, /run in RAM is gone at reboot, /proc /sys /dev are live kernel information, the NFS share stays on stor01. The work always happens in RAM, and WHERE I write it decides if it stays: a live change (sysctl -w, systemd-run, --runtime, mount, ip addr) is a test that is gone at reboot; a file on the disk – usually in /etc – is read at every boot and stays. My test proved it: systemd-run made a temporary box in system.slice with MemoryMax=100M, the OOM killer stopped stress-ng at about 96 MB inside that box only (CONSTRAINT_MEMCG), and the box disappeared when the test ended.